EXP – 5: SCHEDULING ALGORITHMS (SHORTEST JOB FIRST)

rene@localhost:~ | rene@localhost:~ — /usr/bin/vim sjf.c

```c
#include<stdio.h>
int main(){
        int n;
        printf("Enter number of processes: ");
        scanf("%d" ,&n);
        int at[n],bt[n],ct[n],tat[n],wt[n];
        int done[n];
        for(int i=0;i<n;i++){
                printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
                scanf("%d %d",&at[i],&bt[i]);
                done[i] = 0;
        }
        int completed = 0;
        int completionTime = 0;
        float avgtat = 0, avgwt = 0;
        while(completed < n){
                int idx = -1;
                int minBT = 9999;
                for (int i=0;i<n;i++){
                        if(at[i]<=completionTime && done[i] == 0){
                                if (bt[i] < minBT){
                                        minBT = bt[i];
                                        idx = i;
                                }
                        }
                }
                if(idx != -1){
                        if(completionTime<at[idx]){
                                completionTime = at[idx];
                        }
                        completionTime += bt[idx];
                        tat[idx] = completionTime - at[idx];
                        wt[idx] = tat[idx] - bt[idx];
                        done[idx] = 1;
                        completed++;
                }else{
                        completionTime++;
                }
        }
        for(int i =0;i<n;i++){
                avgtat += tat[i];
                avgwt += wt[i];
        }
```

9,16-30 Top

rene@localhost:~ | rene@localhost:~ — /usr/bin/vim sjf.c

```c
        int at[n],bt[n],ct[n],tat[n],wt[n];
        int done[n];
        for(int i=0;i<n;i++){
                printf("Enter Arrival Time and Burst Time for P%d: ",i+1);
                scanf("%d %d",&at[i],&bt[i]);
                done[i] = 0;
        }
        int completed = 0;
        int completionTime = 0;
        float avgtat = 0, avgwt = 0;
        while(completed < n){
                int idx = -1;
                int minBT = 9999;
                for (int i=0;i<n;i++){
                        if(at[i]<=completionTime && done[i] == 0){
                                if (bt[i] < minBT){
                                        minBT = bt[i];
                                        idx = i;
                                }
                        }
                }
                if(idx != -1){
                        if(completionTime<at[idx]){
                                completionTime = at[idx];
                        }
                        completionTime += bt[idx];
                        tat[idx] = completionTime - at[idx];
                        wt[idx] = tat[idx] - bt[idx];
                        done[idx] = 1;
                        completed++;
                }else{
                        completionTime++;
                }
        }
        for(int i =0;i<n;i++){
                avgtat += tat[i];
                avgwt += wt[i];
        }
        printf("\nAverage Turnaround Time: %.2f\n",avgtat/n);
        printf("Average waiting Time: %.2f\n",avgwt/n);
        return 0;
}
```

43,2-9 Bot

```
[rene@localhost ~]$ vi sjf.c
[rene@localhost ~]$ gcc sjf.c
[rene@localhost ~]$ ./a.out
Enter number of processes: 4
Enter Arrival Time and Burst Time for P1: 1 3
Enter Arrival Time and Burst Time for P2: 2 4
Enter Arrival Time and Burst Time for P3: 1 2
Enter Arrival Time and Burst Time for P4: 4 4

Average Turnaround Time: 6.25
Average waiting Time: 3.00
[rene@localhost ~]$
```